

IMAGE PROCESSING

IMAGE POINT TRANSFORMS

Víťa Beran



Brno University of Technology
Faculty of Information Technology
Božetěchova 1/2, 602 00 Brno

- How to change image intensities?
- How to segment image by intensity?
- How to improve image contrast?
- How to change image shape?
- How to stitch images?
- How to combine geometry transforms?
- Why we need image point interpolation?

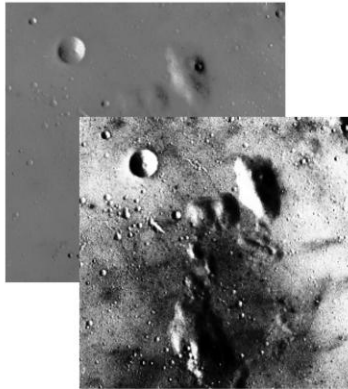
Task – image stitching (mosaicing)



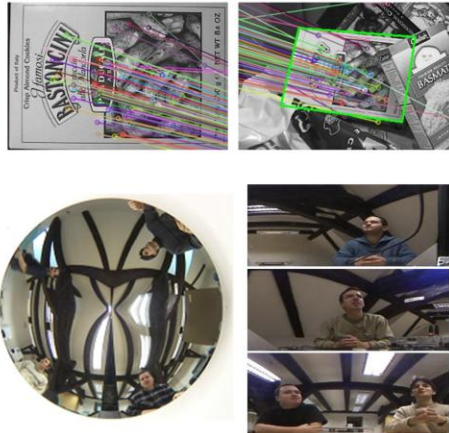


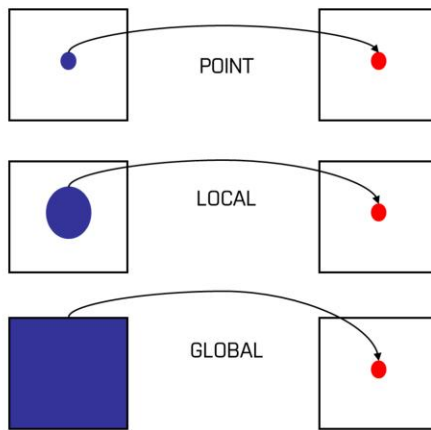


INTENSITY TRANSFORMATION



GEOMETRY TRANSFORMATION







The easiest operation over the image

An intensity transform converts the "old" point value to a "new" value based on a predefined function

It works ONLY with a given point value

Is part of them:

- Intensity operations

- Thresholding

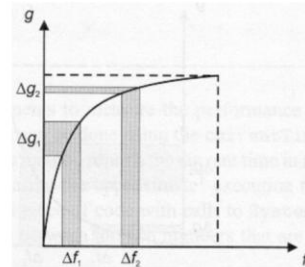
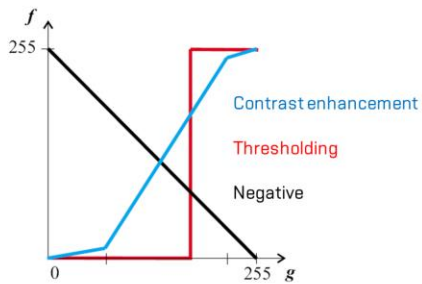
Original



Contrast enhancement



Negativ



E.g. Contrast change

Graph shows the dependency of input intensity to the output intensity

$$I = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$



The intensity (brightness) corresponds to the gray color

Thresholding

T FIT

Original Image

BINARY

BINARY_INV

TRUNC

TOZERO

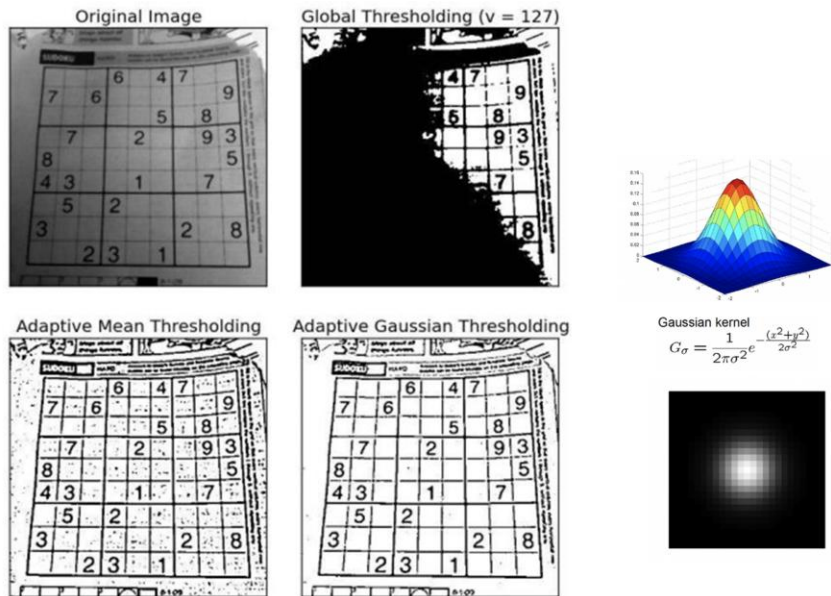
TOZERO_INV

ORIGINAL	OUTPUT	BINARY	BIN_INV	TRUNC	TOZERO	TOZERO_INV
$I(x,y) > T$	$O(x,y) =$	255	0	T	$I(x,y)$	0
$I(x,y) \leq T$	$O(x,y) =$	0	255	$I(x,y)$	0	$I(x,y)$

Image Point Transforms | 10

The threshold input can be a color image or a grayscale image

The result is mostly a binary image representing values above, resp. below the selected threshold.



algorithm calculate the threshold for a small regions of the image.

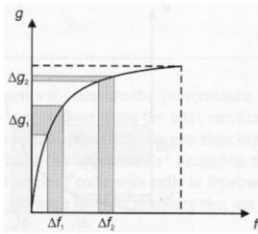
different thresholds for different regions of the same image

better results for images with varying illumination

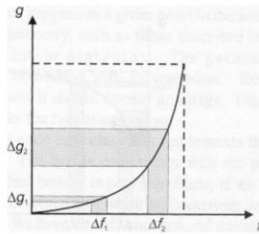
Mean: threshold value is the mean of neighbourhood area.

Gaussian: threshold value is the weighted sum of neighbourhood values where weights are a gaussian window.

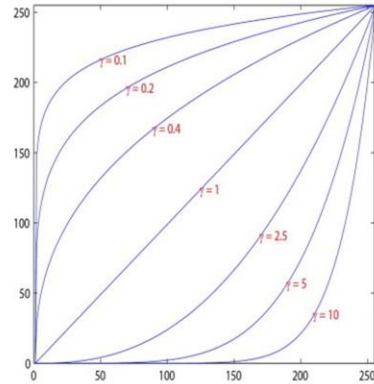
Logarithmic



Exponential



$$O = \left(\frac{I}{255} \right)^\gamma \times 255$$

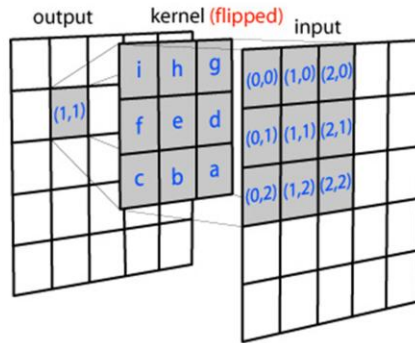


Logarithmic

It serves to increase detail in dark areas of the image and reduce detail in light areas

Exponencial

Inverse to the previous example



$$g(x, y) = \sum_{i=-k}^k \sum_{j=-l}^l f(x-i, y-j) \cdot h(i, j)$$

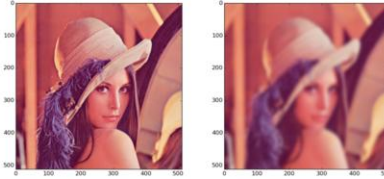
The new point value is estimated from a small area
 Surrounding is defined as a window, e.g. 3x3 points
 There are problems at the edges of the image – mask overlap
 An example is a 2D convolution.

The contribution of individual points in the neighborhood is weighted in a linear combination of the coefficients of the matrix **h** - called the mask or the convolutional kernel

More detail about convolution in other lectures about filtering, edge detection etc.

- Averaging around 3x3 point.

$$h = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



- Different variants of point highlighting.

$$h = \frac{1}{10} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix}, \quad h = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$



HISTOGRAM ??

HISTOGRAM ??

0	0	1	0	2	0
1	0	7	7	7	0
0	7	0	0	7	0
1	0	0	7	2	0
0	0	7	1	0	1
1	0	7	7	7	0

frequencies

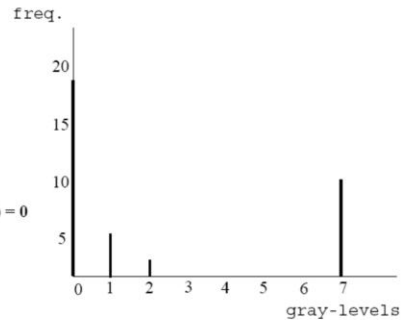
$f(0) = 18$

$f(1) = 6$

$f(2) = 2$

$f(3) = f(4) = f(5) = f(6) = 0$

$f(7) = 10$



Global operations use information from the whole image

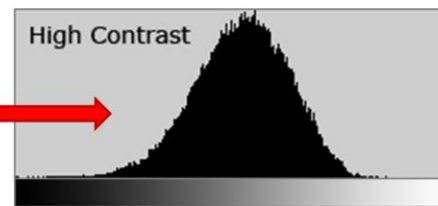
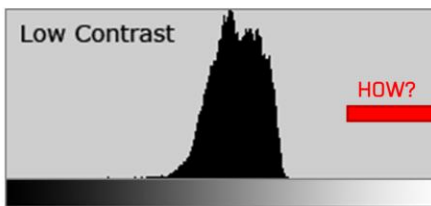
The resulting information is used to change the values of the individual points in the image

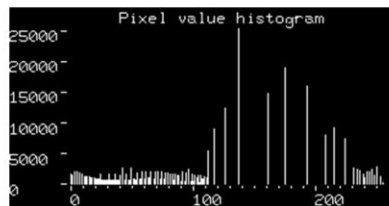
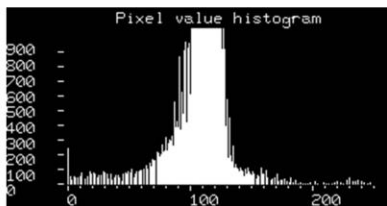
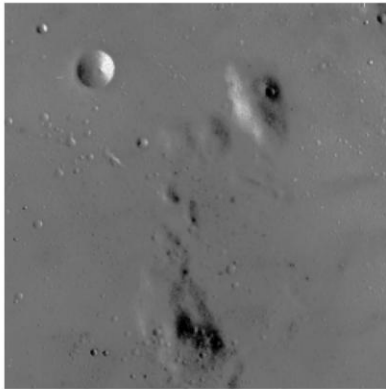
Examples are: Histogram equalization or Contrast enhancement

Histogram - it is a statistical representation of the image

Represents a graph of the frequency of occurrence of individual colors (shades, intensities ...) in the image

Used to evaluate image quality





Histogram is a graphical representation of a vector with a number of components equal to the number of possible levels of pixel intensities and serves to evaluate the distribution of intensities in the digital image.

Equalizing the histogram is an algorithm that changes the intensity distribution in the image to the extent that it occurs within a wide range, at approximately the same frequency.

Equalization algorithms are based on statistical methods.

Increase the contrast by fully using the luminance scale

Brightness normalize (for example, for automatic comparison)

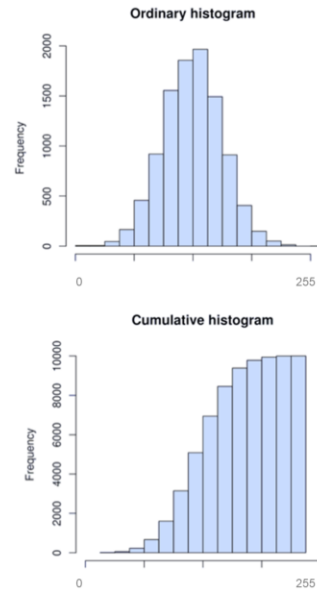
- Cumulative histogram

$$c(i) = \sum_{j=0}^i p(j)$$

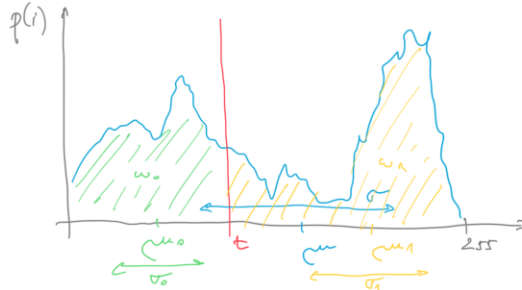
- 255 histogram levels
- Image size $M \times N$

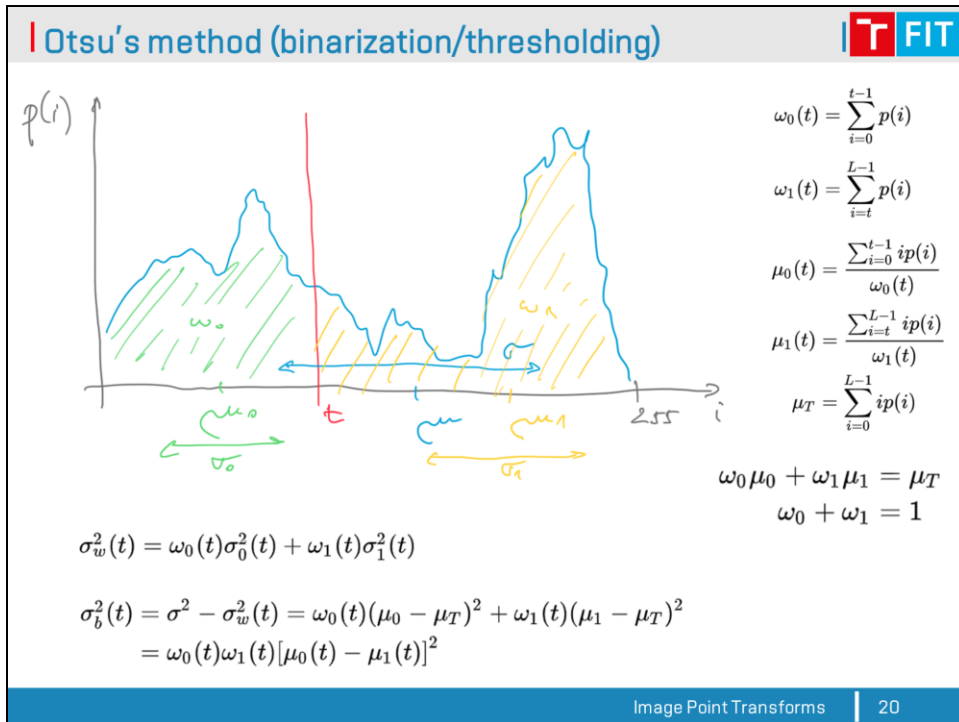
- Output intensity

$$g(i) = \frac{c(i)}{M \times N} 255 = c(i) \frac{255}{M \times N}$$



- Randomly selected threshold T
- Iterate
 - Split to 2 sets: foreground/background points
 - $G1 = \{(x,y) : I(x,y) > T\}$
 - $G2 = \{(x,y) : I(x,y) \leq T\}$
 - Average values of each set
 - $m1 = \text{mean}(G1)$
 - $m2 = \text{mean}(G2)$
 - New threshold
 - $T = 1/2 * (m1 + m2)$



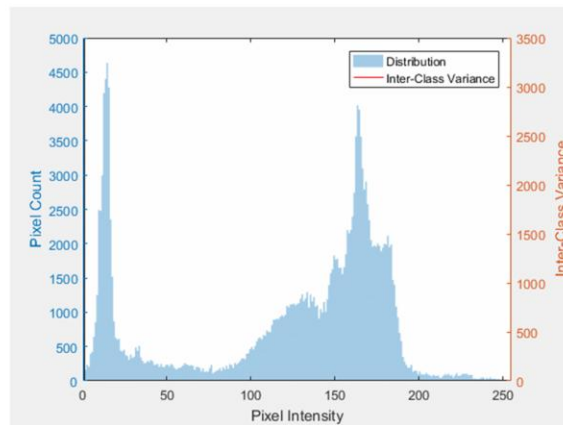


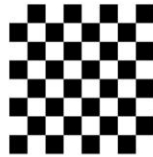
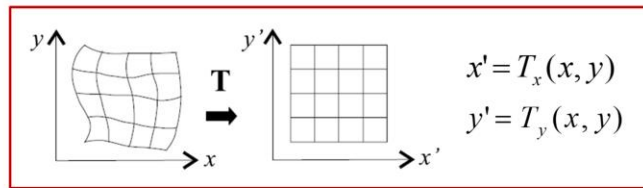
The algorithm exhaustively searches for the threshold that minimizes the intra-class variance, defined as a weighted sum of variances of the two classes (σ_w^2).

For 2 classes, minimizing the intra-class variance is equivalent to maximizing inter-class variance (σ_b^2).

Problem: if the object area is small compared with the background area.

Improvements: two-dimensional Otsu's method – pairs of pixel value and the average of its neighborhood and works with the joint probability mass function in a 2-dimensional histogram.

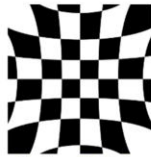




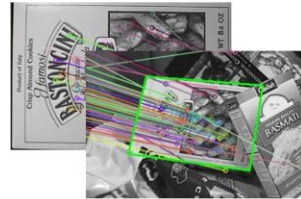
No distortion



Positive radial distortion
(Barrel distortion)



Negative radial distortion
(Pincushion distortion)



Changes the image geometry.

Affine transformations: Scale change, translation, rotation, skew of 2D image (i.e. by known transformation)

Removing geometric lens/camera distortions.

Log-polar transformation.

Image interpolations – why?

Geometric operations map the point in the image

from coordinates x_1, y_1

to the output image at the new position x_2, y_2

Affine transformations




Translate

$$\begin{aligned} x_2 &= x_1 + t_x \\ y_2 &= y_1 + t_y \end{aligned}$$



Skew

$$\begin{aligned} x_2 &= x_1 + y_1 \cdot sh_x \\ y_2 &= x_1 \cdot sh_y + y_1 \end{aligned}$$



Scale

$$\begin{aligned} x_2 &= x_1 s_x \\ y_2 &= y_1 s_y \end{aligned}$$



Rotate

$$\begin{aligned} x_2 &= x_1 \cos \alpha - y_1 \sin \alpha \\ y_2 &= x_1 \sin \alpha + y_1 \cos \alpha \end{aligned}$$

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = A \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} + B$$

Image Point Transforms
23

Why affine?

Lines that are straight and parallel remain the same even after the transformation

These operations include:

Scaling, Rotation, Mirroring, Translation

These operations can be described by the following function:

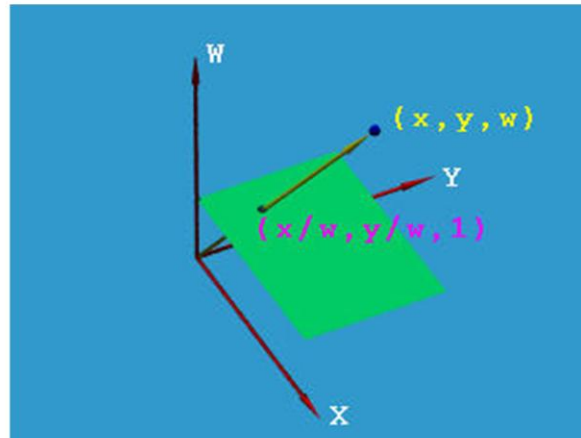
Translation - Performs a change in the position of each of the points for that section in the x-axis and in the y-axis.

Scale - Multiply the coordinate of each point with a scaling factor in the x and y axis

Rotation - Rotation for both coordinates must be performed by the selected angle α

- Rotation is performed around the origin of the coordinate system
- For rotation around its axis, the image is shifted, rotated around the

beginning, and then moved back



$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = A \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} + B$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} a_{00} & a_{01} & t_x \\ a_{10} & a_{11} & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}$$

They are commonly used in projective geometry, computer graphics, robotics, and so on.

They allow to express basic linear transformations or perspective projections using a single matrix.

Box $[x, y]^T$ in Cartesian coordinates is represented in homogeneous coordinates by the vector $[X, Y, W]^T$ and holds:

$$[x, y]^T = [X/W, Y/W]^T$$

where $W = 1$ for representation of points in 2D Euclidean space and $W = 0$ for vectors.



Translate

$$\begin{aligned}x_2 &= x_1 + t_x \\ y_2 &= y_1 + t_y\end{aligned}$$

Skew

$$\begin{aligned}x_2 &= x_1 + y_1 \cdot sh_x \\ y_2 &= x_1 \cdot sh_y + y_1\end{aligned}$$

Scale

$$\begin{aligned}x_2 &= x_1 s_x \\ y_2 &= y_1 s_y\end{aligned}$$

Rotate

$$\begin{aligned}x_2 &= x_1 \cos \alpha - y_1 \sin \alpha \\ y_2 &= x_1 \sin \alpha + y_1 \cos \alpha\end{aligned}$$

$$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Geometric operations map the point in the image

from coordinates x_1, y_1

to the output image at the new position x_2, y_2

These operations include:

Scaling, Rotation, Mirroring, Translation

These operations can be described by the following function:

Translation - Performs a change in the position of each of the points for that section in the x-axis and in the y-axis.

Scale - Multiply the coordinate of each point with a scaling factor in the x and y axis

Rotation - Rotation for both coordinates must be performed by the selected angle α

- Rotation is performed around the origin of the coordinate system
- For rotation around its axis, the image is shifted, rotated around the beginning, and then moved back

- it depends on the order of applying transforms
 - shift and rotation vs. rotation and shift
- resulting transformation as a result of multiplication of partial transformation matrices
- transformation matrix for rotation by an angle α around the point C
 - Shift rotated point P using inverse transform T_C^{-1}
 - Rotate the point by given angle α using rotation matrix R_α
 - Shift back the point using matrix T_C

$$P' = T_C \cdot R_\alpha \cdot T_C^{-1} \cdot P$$

$$M = T_C \cdot R_\alpha \cdot T_C^{-1}$$

$$P' = M \cdot P$$

- Point $P = (x_P, y_P, 1)$, center of rotation $C = (x_C, y_C, 1)$

- Translation: $T(t_x, t_y) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix}$

- Rotation: $R(\alpha) = \begin{bmatrix} \cos \alpha & \sin \alpha & 0 \\ -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}$

- Rotation of the point P around point C by angle α

$$P' = P \cdot T_C^{-1} \cdot R_\alpha \cdot T_C$$

- Point $P = (x_P, y_P, 1)^T$, center of rotation $C = (x_C, y_C, 1)^T$

- Translation: $T(t_x, t_y) = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$

- Rotation: $R(\alpha) = \begin{bmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}$

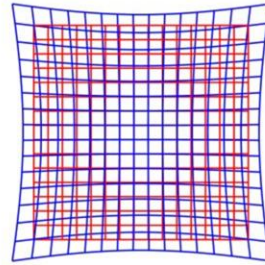
- Rotation of the point P around point C by angle α

$$P' = T_C \cdot R_\alpha \cdot T_C^{-1} \cdot P$$

RADIAL DISTORTION

$$x_{distorted} = x(1 + k_1r^2 + k_2r^4 + k_3r^6)$$

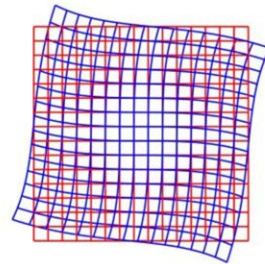
$$y_{distorted} = y(1 + k_1r^2 + k_2r^4 + k_3r^6)$$



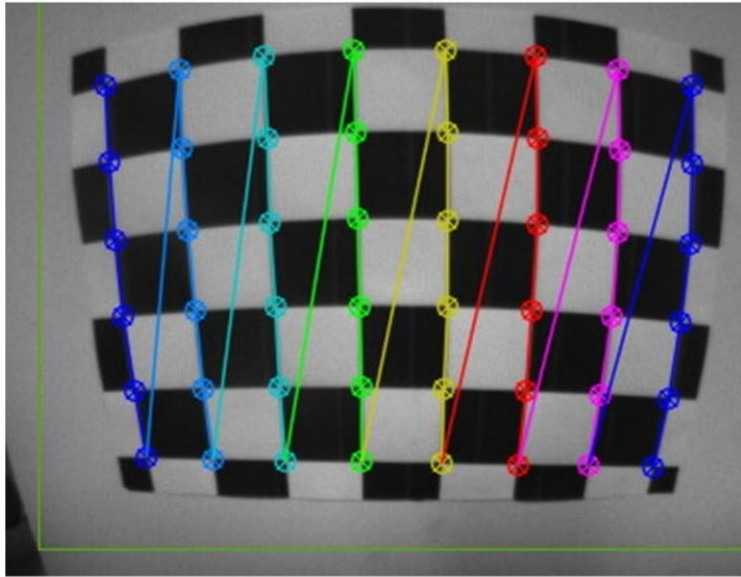
TANGENTIAL DISTORTION

$$x_{distorted} = x + [2p_1xy + p_2(r^2 + 2x^2)]$$

$$y_{distorted} = y + [p_1(r^2 + 2y^2) + 2p_2xy]$$

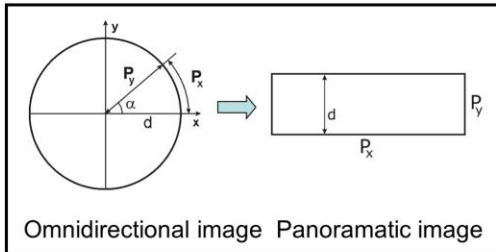


Some pinhole cameras introduce significant distortion to images. Two major kinds of distortion are radial distortion and tangential distortion.



$$\rho = \log(\sqrt{(x - x_c)^2 + (y - y_c)^2})$$

$$\theta = \text{atan2}(y - y_c, x - x_c)$$



The log-polar (**logarithmic polar coordinates**) transform is performed by remapping points from the 2D Cartesian coordinate system (x, y) to the 2D log-polar coordinate system (ρ, θ)

The **polar coordinate system** is a two-dimensional coordinate system in which each point on a plane is determined by a distance from a reference point and an angle from a reference direction.



PROBLEM ?

$$S = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Transformation of point coordinates

the new position of each point is calculated in continuous coordinates (real numbers) $x' = T(x)$

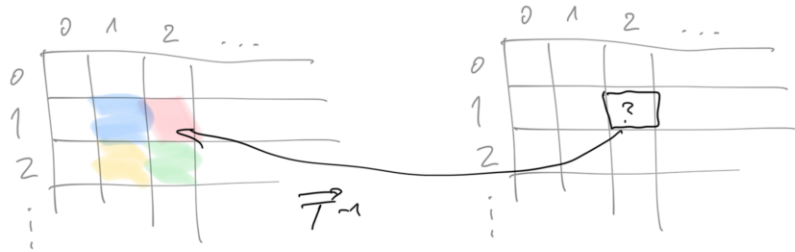
Appropriate to use image approximation

From non-integer positions x', y' we look for the value of brightness in integer positions.

Problems:

Sometimes x' can be out of the picture.

Complex calculation for the approximate value of unordered data.



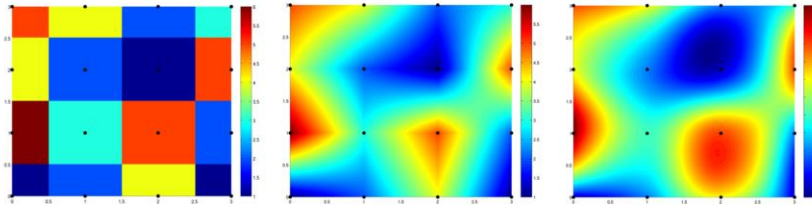
$$S^{-1} = \begin{bmatrix} \frac{1}{3} & 0 & 0 \\ 0 & \frac{1}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The coordinates of the point (x, y) in the input image can be calculated by inverting the relation $(x', y') = T(x, y)$

i. e., $(x, y) = T^{-1}(x', y')$

Interpolation - bilinear, trilinear, bicubic

- Nearest neighbor method
- Bilinear interpolation
- Bicubic interpolation
- Spline interpolation



Use to obtain an image value value beyond measured discrete values.

Commonly used methods:

Nearest neighbor method

Bilinear interpolation

loss of sharpness

Bicubic interpolation

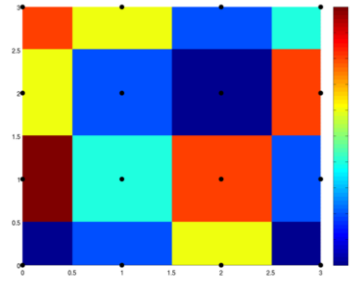
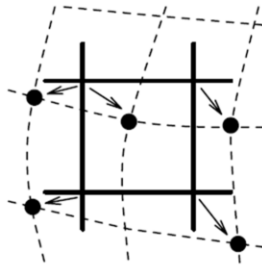
achieves better sharpness than bilinear,

color artifacts may arise

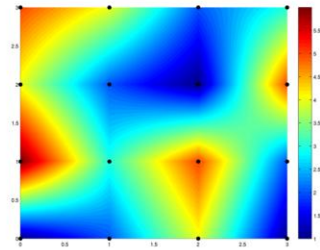
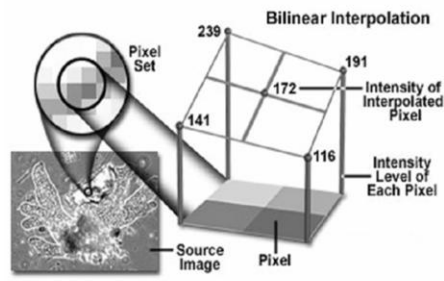
Spline interpolation

In the case of a color image, we perform interpolation for all three color components separately.

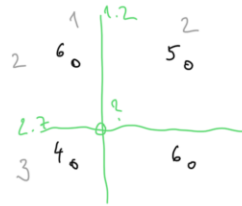
- Assigns the point (x, y) to the brightness value of the closest point in the discrete grid.
- The advantage is speed and color stability.



- It is done from 4 points in the neighborhood (x, y)
- The influence of each point decreases with the distance



- $g(x, y)$ is the output image and $f(l, k)$ is the input image



$$l = \text{round}(x), k = \text{round}(y)$$

$$a = x - l, b = y - k$$

$$g(x, y) = (1-a)(1-b)f(l, k) + a(1-b)f(l+1, k) + b(1-a)f(l, k+1) + abf(l+1, k+1)$$

$$a = x - l = 0.2 \quad ; \quad b = y - k = 0.7$$

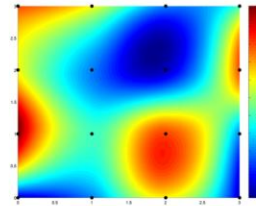
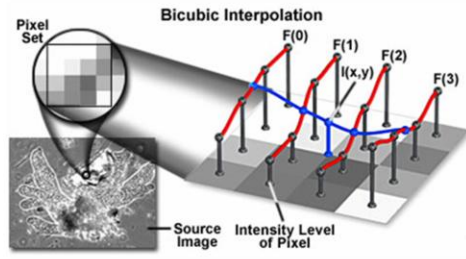
$$g(1.2, 2.7) = (1-0.2)(1-0.7) \cdot 60 + (0.2)(1-0.7) \cdot 50 + (1-0.2)(0.7) \cdot 30 + (0.2)(0.7) \cdot 60 =$$

$$= 0.24 \cdot 60 + 0.06 \cdot 50 + 0.56 \cdot 30 + 0.14 \cdot 60 =$$

$$= \underline{\underline{4.82}}$$

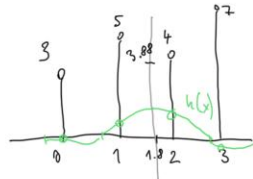
$$\sum = 1$$

- Based on the local interpolation of the pixelated function by a 16-point polynomial.



- Example of an interpolation core

$$h_3 = \begin{cases} 1 - 2|x|^2 + |x|^3 & \text{pro } 0 \leq |x| < 1, \\ 4 - 8|x| + 5|x|^2 - |x|^3 & \text{pro } 1 \leq |x| < 2, \\ 0 & \text{jinde.} \end{cases}$$

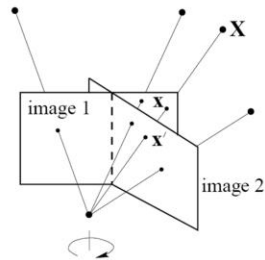


$$f(x) = \sum_i I(i) \cdot h(x-i)$$

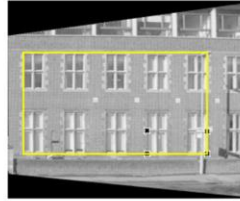
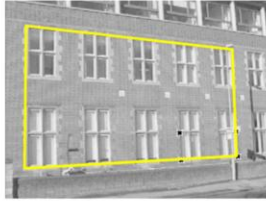
$$\begin{aligned} f(1.8) &= I_0 \cdot h(1.8) + I_1 \cdot h(0.8) + I_2 \cdot h(0.2) + I_3 \cdot h(1.2) = \\ &= 3 \cdot (-0.032) + 5 \cdot (0.232) + 4 \cdot (0.928) + 7 \cdot (-0.128) = \\ &= \underline{\underline{3.88}} \end{aligned}$$

Σ = 1

- Projectivity, collineation, projective transformation
- Maps points from a straight line again to points on a straight line
- Expresses transformation between images



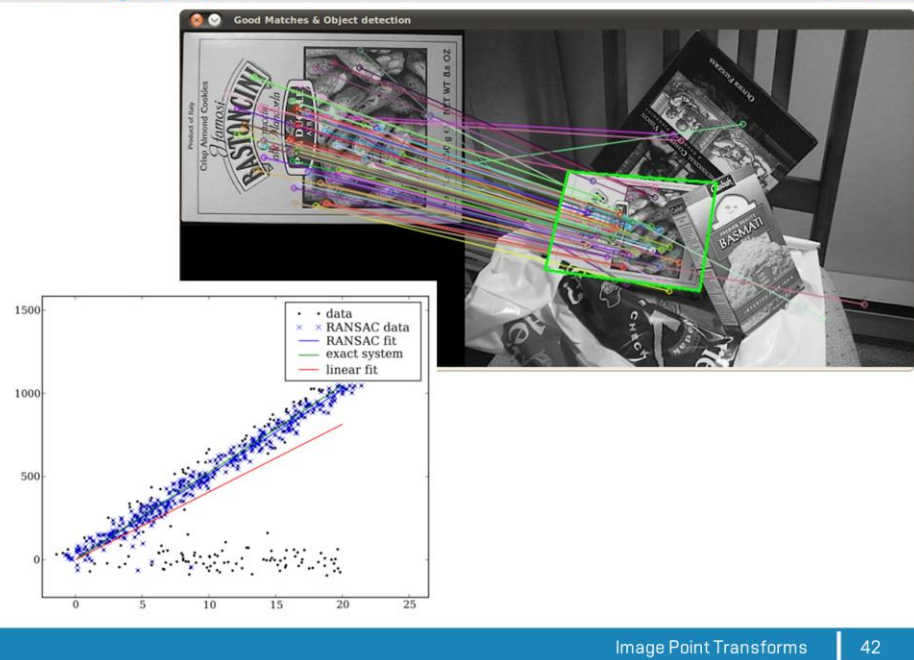
• Linear transformation 3D homogenous vectors



from Hartley & Zisserman

$$\begin{bmatrix} x'_1 \\ x'_2 \\ x'_3 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

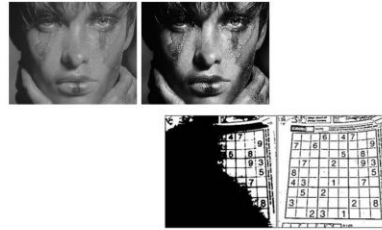
$$\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 & -x'_1 \cdot x_1 & -x'_1 \cdot y_1 \\ 0 & 0 & 0 & x_1 & y_1 & 1 & -y'_1 \cdot x_1 & -y'_1 \cdot y_1 \\ x_2 & y_2 & 1 & 0 & 0 & 0 & -x'_2 \cdot x_2 & -x'_2 \cdot y_2 \\ 0 & 0 & 0 & x_2 & y_2 & 1 & -y'_2 \cdot x_2 & -y'_2 \cdot y_2 \\ & & & & \vdots & & & \\ x_n & y_n & 1 & 0 & 0 & 0 & -x'_n \cdot x_n & -x'_n \cdot y_n \\ 0 & 0 & 0 & x_n & y_n & 1 & -y'_n \cdot x_n & -y'_n \cdot y_n \end{bmatrix} \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \end{bmatrix} = \begin{bmatrix} x'_1 \\ y'_1 \\ x'_2 \\ y'_2 \\ \vdots \\ x'_n \\ y'_n \end{bmatrix}$$



If we have too many points - the least squares method or RANSAC

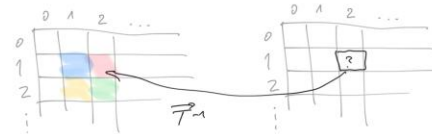
Intensity transformations

- Point, local, global
- Histogram equalization
- Thresholding (adaptive)



Geometry transformation

- Composition (affine transf.)
- Inverse transformation
- Image interpolation



$$P' = T_C \cdot R_\alpha \cdot T_C^{-1} \cdot P$$

$$M = T_C \cdot R_\alpha \cdot T_C^{-1}$$

$$P' = M \cdot P$$

